An Automated Rule-Based Fault Detection Method for Rotating Machines Using Accelerometer Vibration Signal Analysis

Fatema Zerin Prottasha and Mohammad Mahmudul Hasan
Department of Electrical and Electronic Engineering
Islamic University of Technology (IUT), Gazipur, Bangladesh

***Abstract*—Although vibration-based condition monitoring is widely adopted for rotating machinery, fault diagnosis conventionally relies on manual spectral inspection by domain experts. This paper proposes an automated, rule-based fault detection framework that identifies mechanical faults directly from accelerometer vibration signals without human intervention. The method compares the vibration profile of a machine under test against a healthy baseline by extracting four key diagnostic features: root-mean-square (RMS) energy ratio, shaft rotational order (×RPM), spectral peak count via fast Fourier transform (FFT), and peak frequency/phase matching indices. An empirical rule base maps these features to classify bearing faults (inner race, outer race, and ball bearing), mechanical anomalies (unbalance and misalignment), and gear defects. Implemented in MATLAB with an intuitive graphical interface, the proposed system automatically declares the fault category and outputs supporting statistical evidence. Experimental validation on a gear fault dataset confirms accurate, autonomous classification (RMS ratio 0.597, peak order 0.2×RPM), demonstrating its efficacy for practical, real-time machine health monitoring.**

***Index Terms*—Condition monitoring, fault detection, rotating machines, induction motors, vibration analysis, accelerometer, fast Fourier transform, order analysis, bearing fault, gear fault, MATLAB.**

I. Introduction

Rotating machines, and induction motors in particular, are used in almost every industrial process. A developing fault that remains undetected can escalate into a massive machine failure, causing loss of production, repair costs, and, in critical installations, risk to human life. Reliable and timely fault detection is therefore essential for modern maintenance practice.

Historically, machine faults were diagnosed by expert systems through *off-line fault detection*, in which experienced specialists manually assessed the overall condition of a machine and judged whether a particular fault was likely to occur. Such an approach cannot guarantee authentic results because it depends on subjective human analysis; moreover, experts in the maintenance of different machine types are not always available. These shortcomings motivated the introduction of analytical methods based on real-time monitoring, including vibration monitoring, acoustic emission monitoring, air-gap eccentricity monitoring, partial discharge monitoring, magnetic flux monitoring, current monitoring, and voltage monitoring.

Among these techniques, current, voltage, magnetic flux, and partial discharge monitoring primarily reveal *electrical* faults, and the associated measurement equipment is highly expensive. Vibration analysis, in contrast, is generalized for a number of fault types, is economically efficient, and yields reliable data on which the precise detection of a particular fault type can depend. Since all rotating machines produce noise and vibration of conspicuous amplitude, originating from magnetic, mechanical, and aerodynamic sources, several types of faults can be differentiated using vibration parameters alone. Vibration and noise are an inevitable signature of any machine and thus constitute a robust indicator of both the operating phase and the health condition of the machine in real time.

The importance of mechanical faults is confirmed by reliability statistics. According to IEEE data [2], bearing faults account for 44% of induction motor failures, followed by stator faults (26%), rotor faults (8%), and other faults (22%). Data reported by EPRI [3] show a similar trend, with bearing faults (41%), stator faults (36%), rotor faults (9%), and other faults (14%), as illustrated in Fig. 1. Most failures therefore originate from mechanical deficiencies.

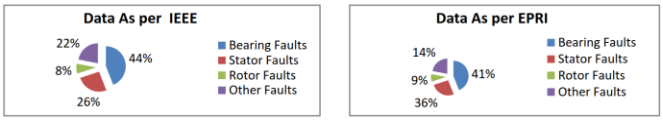


Fig. 1. Percentage distribution of failed components in induction motors according to IEEE (left) and EPRI (right) reliability data [2], [3].

Although vibration signal processing in the time domain, frequency domain, power spectral density, and other representations has been used extensively by experts, the faults themselves are still detected *manually*: a specialist must inspect the processed graphs and infer the fault type. Manual detection, even when supported by electromagnetic sensing devices, cannot always predict machine faults and failures in an authentic and impeccable manner. This gap motivates the present work, whose main contributions are as follows:

- A complete processing chain that compares the accelerometer signal of a machine under test with that of a healthy machine using time-domain RMS analysis, FFT-based spectral analysis, peak order analysis, peak counting, and peak frequency and phase matching.
- An empirically derived parametric table and rule base that relate these vibration features to inner race, outer race,

and ball bearing faults, mechanical faults (unbalance, horizontal and vertical misalignment), and gear faults.

- An automated decision stage in which the computer, rather than a human expert, declares the fault type, together with a user-oriented MATLAB interface that reports the diagnosis and its supporting evidence.

The remainder of this paper is organized as follows. Section II reviews machine fault types and vibration analysis techniques. Section III describes the proposed method. Section IV presents the software implementation. Section V reports a representative case study, Section VI discusses the significance and limitations of the method, and Section VII concludes the paper.

II. Background

A. Taxonomy of Rotating Machine Faults

Faults in rotating electrical machines can be broadly divided into electrical and mechanical faults, as shown in Fig. 2. Electrical faults include stator faults (e.g., stator winding faults) and rotor faults (e.g., broken rotor bars). Mechanical faults comprise bearing faults, which are further subdivided into inner race, outer race, and ball bearing faults; gear faults; and unbalance faults, caused by weight imbalance or deviation (misalignment). Since this work relies on vibration data, its scope is restricted to mechanical faults.

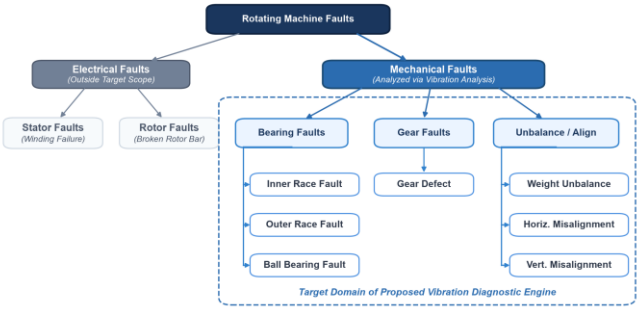


Fig. 2. Classification of faults in rotating machines. This work addresses the mechanical branch: bearing (inner race, outer race, ball bearing), gear, and unbalance faults.

B. Vibration Acquisition and Analysis

Vibration is sensed by accelerometers installed at the fault-prone locations of the machine under test. When the machine is in operation, the accelerometers record the localized vibration as a time series. Various analysis techniques can be applied to such acceleration data, each with characteristic units and purposes, as summarized in Fig. 3 [1]. Time-domain representations (acceleration, interval average, interval RMS, and trimmed-mean acceleration versus time) characterize the variation of acceleration magnitude, whereas frequency-domain representations (power spectral density, cumulative RMS, and RMS acceleration versus frequency) quantify the distribution of vibration energy over frequency, and the spectrogram reveals how the signal varies in both time and frequency.

Analysis Technique	*Units*	*Data Typically Used*	*Use*
Acceleration versus Time	g or fraction of g	OARE SAMS	precise accounting of variation of acceleration magnitude as a function of time
Interval Average Acceleration versus Time	g or fraction of g	SAMS	indication of net accelerations lasting for time period ≥ interval parameter
Interval RMS Acceleration versus Time	g or fraction of g	SAMS	measure of oscillatory content in data
Trimmean Acceleration versus Time	g or fraction of g	OARE	smooth data and reject transient, higher magnitude contributions
Quasi-steady Three-dimensional Histogram	percentage of time and g or fraction of g	OARE	summarize acceleration vector magnitude and direction
Mapping of OARE Data	g or fraction of g	OARE	predict quasi-steady environment at locations other than OARE sensor
Power Spectral Density versus Frequency	g^2/Hz	SAMS	estimate of distribution of energy with respect to frequency
Cumulative RMS Acceleration versus Frequency	g_{RMS}	SAMS	quantifies contributions of spectral components to overall RMS acceleration level for time period studied in cumulative fashion
RMS Acceleration versus Frequency	g_{RMS}	SAMS	quantifies contributions of spectral components per frequency bin
Spectrogram	g^2/Hz	SAMS	road map of how acceleration signals vary with respect to both time and frequency

Fig. 3. Acceleration data analysis techniques, their units, typical data sources, and uses (reproduced from [1]).

In general, vibration data analysis can be categorized as follows:

1) RMS of vibration amplitude;
2) time-domain analysis;
3) frequency analysis (FFT);
4) time-frequency analysis; and
5) higher-order statistics.

Fault detection methods can likewise be categorized into model-based approaches, trending, fault thresholds, and multi-dimensional space techniques. Regardless of the chosen method, vibration data sampled from the factory-original (healthy) machine should be recorded as a reference, so that vibration variables obtained later can be compared against it to determine whether a fault has already occurred or whether the machine is at risk. The method proposed in this paper follows this comparative principle and combines RMS, time-domain, and FFT analysis with a fault-threshold decision strategy.

III. Proposed Methodology

A. Overview

The proposed method takes two accelerometer recordings as input: a reference signal $a[n]$ from a healthy machine and a test signal $x[n]$ from the machine under test. Both signals are analyzed in the time and frequency domains, a set of comparative features is extracted, and the features are evaluated against a rule base to declare the fault type. The complete procedure is summarized in Algorithm 1.

Algorithm 1: Automated vibration-based fault detection

Input: healthy signal $a[n]$, test signal $x[n]$, sampling frequency F_s, optional duration T

Output: declared fault type(s) and supporting feature report

1: Read the healthy and test accelerometer data.

2: Determine the analysis length L and duration T; align both signals.

3: Perform time-domain analysis (vibration amplitude vs. time).
4: Compute the RMS values of both signals and their ratio R.
5: Compute the FFT and the single-sided amplitude spectra.
6: Calculate the RPM order M at which the test signal exhibits fault symptoms.
7: Count the significant spectral peaks of both spectra and compare them.
8: Match the frequency and serial position (phase) of healthy and faulty peaks.
9: Evaluate the fault conditions derived from the parametric table (Table I).
10: Correlate features and conditions to declare the exact fault type(s).
11: Display the diagnosis, the feature report, and comparative plots.

B. Data Acquisition and Pre-processing

The healthy and test recordings, stored as MATLAB data files, are loaded together with the sampling frequency F_s. If the recording duration T is known, both signals are truncated to $L = F_s T$ samples. Otherwise, the analysis length is taken as the length of the shorter recording, $L = \min(L_a, L_x)$, and the duration is calculated as $T = L/F_s$. A common time axis $t_n = n/F_s$, $n = 0, 1, \ldots, L-1$, is then constructed so that the two signals are compared over identical intervals.

C. Time-Domain RMS Analysis

The overall vibration energy of each signal is quantified by its RMS value,

$$RMS_a = \sqrt{[(1/L)\,\Sigma_n\, a^2[n]]}, \quad RMS_x = \sqrt{[(1/L)\,\Sigma_n\, x^2[n]]} \quad (1)$$

and the relative severity of the test signal is expressed by the RMS ratio

$$R = RMS_x / RMS_a \quad (2)$$

A test signal whose RMS exceeds that of the healthy reference ($R > 1$) is flagged as a potentially severe fault, a condition that is subsequently required for the detection of bearing faults.

D. Frequency-Domain Analysis

The discrete Fourier transform of each signal is computed using the FFT,

$$X[k] = \Sigma_n\, x[n]\, e^{\wedge}(-j2\pi kn/L), \quad k = 0, 1, \ldots, L-1 \quad (3)$$

from which a scaled two-sided spectrum $P_2[k] = 10|X[k]|/L$ and the single-sided spectrum are obtained:

$$P_1[k] = P_2[k]\ (k = 0, L/2); \quad 2P_2[k]\ (else) \quad (4)$$

with frequency bins $f_k = kF_s/L$, $k = 0, 1, \ldots, L/2$. The same procedure applied to the healthy signal yields its spectrum $P_a[k]$.

E. Peak Order Analysis

Fault mechanisms in rotating machines excite vibration components at characteristic multiples (orders) of the shaft rotational speed. To capture this behavior, the order spectrum of each signal is computed, and the two most prominent peak orders of the healthy spectrum ($o_{a,1}$, $o_{a,2}$) and of the test spectrum ($o_{x,1}$, $o_{x,2}$) are identified. The order at which the test signal exhibits fault symptoms is quantified as the mean ratio

$$M = \tfrac{1}{2}\, \Sigma_{i=1}^{2}\, (o_{x,i} / o_{a,i}) \quad (5)$$

which is reported in multiples of the shaft speed (×RPM).

F. Spectral Peak Detection and Counting

Significant spectral peaks are detected using a local-maxima peak finder that ignores small peaks. A common adaptive threshold, derived from the maximum of the healthy spectrum,

$$\tau = max_k\, P_a[k] / \pi \quad (6)$$

is applied to both spectra, yielding N_a healthy peaks and N_x faulty peaks. The peaks are then ranked in descending order of amplitude, and the frequency of each ranked peak ($f_{a,i}$ and $f_{x,j}$) is recorded. A change in the number of significant peaks indicates a redistribution of vibration energy caused by the fault.

G. Peak Frequency and Phase Matching

To determine whether the test signal preserves the spectral structure of the healthy machine, the ranked peak frequencies are compared after rounding to the nearest hertz. Two counts are computed: the number of matching peak frequencies irrespective of rank,

$$C_f = \Sigma_i\, \Sigma_j\, 1[round(f_{a,i}) = round(f_{x,j})] \quad (7)$$

and the number of peaks that match both in frequency and in serial (rank) position,

$$C_e = \Sigma_i\, 1[round(f_{a,i}) = round(f_{x,i})] \quad (8)$$

where $1[\cdot]$ denotes the indicator function. C_e serves as a phase-match measure: bearing faults typically destroy the correspondence between healthy and faulty peaks ($C_f = C_e = 0$), whereas mechanical faults largely preserve it.

H. Parametric Characterization of Fault Types

Through experimental statistics and test drives, the behavior of the extracted features was characterized for each fault type considered. The resulting parametric table, which forms the knowledge base of the method, is presented in Table I. All parameters are based on the vibration properties and output of rotating machines (motors).

TABLE I
VIBRATION FEATURE RANGES FOR DIFFERENT FAULT TYPES

Fault type	Sub-type	Peak phase match	Order (×RPM)	FFT peak number difference	Relative RMS
Bearing	Inner race	0	2.4–18.5	Feasible	> 2×
	Outer race	0	3.5–21	Feasible	> 3.5×
	Ball bearing	0	2–19	Not feasible	≈ 2×
Mechanical	Unbalance	1–2	0.5–1	Almost same	≈ 1
	Horizontal misalignment	0–3	1–2.6	Almost same	≈ 1
	Vertical misalignment	0–3	0.5–2.3	Slightly feasible	≈ 1
Gear	—	0 or 1–2	≤ 0.7	0	Lower

I. Rule-Based Fault Declaration

The ranges in Table I were translated into the decision rules listed in Table II, which are evaluated on the features R, M, N_a, N_x, C_f, and C_e. A bearing fault is first confirmed when the test

signal is more energetic than the healthy reference, none of the peaks match, the peak order exceeds 2×RPM, and the rounded RMS ratio is at least two; the bearing sub-type is then resolved from the specific order and RMS ratio ranges. Mechanical faults are identified by nearly unchanged RMS levels at low orders, whereas gear faults are associated with a reduced RMS level at very low orders, or with the absence of significant peaks. Because the rules are evaluated independently, the method can report multiple concurrent faults; if no rule is satisfied, the machine is declared free of known faults.

TABLE II
DECISION RULES OF THE AUTOMATED FAULT DETECTOR

Declared fault	Decision condition
Bearing fault (prerequisite)	$RMS_x > RMS_a \wedge C_f = 0 \wedge C_e = 0 \wedge M > 2 \wedge \text{round}(R) \geq 2$
Ball bearing	Bearing $\wedge$ round(R) = 2
Inner race	Bearing $\wedge$ $(2 \leq M \leq 3 \vee 10 \leq M \leq 18) \wedge \text{round}(R) \geq 4$
Outer race	Bearing $\wedge$ $(3 \leq M \leq 4 \vee M \geq 18.8) \wedge 3 \leq R \leq 5.5$
Mechanical (unbalance / misalignment)	$0.1 \leq M \leq 2.5 \wedge 0.98 \leq R \leq 1.1$
Gear	$(M < 0.7 \wedge R < 0.7) \vee (N_a = 0 \wedge N_x = 0)$
No known fault	None of the above conditions satisfied

When a mechanical fault is declared, the method additionally advises the user to examine the data of the other sensors to discriminate between unbalance and misalignment, since these sub-types share similar single-axis signatures (Table I).

IV. SOFTWARE IMPLEMENTATION

The method was implemented in MATLAB using a main script and four modular functions: *detectVibrationFeatures*, which performs the time- and frequency-domain analysis, peak order calculation, and peak detection; *computeSingleSidedFFT*, which computes the single-sided spectrum; *extractDominantOrders*, which extracts the dominant peak orders from the order spectrum; and *extractRankedMaxima*, which ranks the detected peaks and returns their amplitudes and frequencies. The complete source code is provided in the Appendix.

Fault detection code is usually written for programmers and is inconvenient for a general user, who would otherwise need to locate the relevant lines, provide inputs, and declare internal variables. The implementation was therefore designed to be as user-friendly as possible. On the input side, the user only (i) selects the healthy and test data files through a graphical file dialog while the code runs in the background, (ii) enters the sampling frequency, and (iii) specifies whether the recording duration is known; if so, any duration that complies with the length of the vibration record may be entered, and otherwise the duration is computed automatically from the record length and sampling frequency.

On the output side, after all vibration data have been processed, the program (i) directly declares the detected fault type, (ii) summarizes all the information that led to the decision in a single information box, eliminating the need to inspect the command window or workspace variables, and (iii) generates comparative and superimposed time- and frequency-domain plots so that the user can also assess the machine condition manually through graphical representation.

V. CASE STUDY AND RESULTS

The operation of the method is illustrated with a test record exhibiting a gear fault. Fig. 4 shows the automatically generated analysis plots. In the time domain, the faulty vibration exhibits a lower amplitude envelope than the healthy vibration. In the frequency domain, the healthy spectrum contains several prominent components, whereas the energy of the faulty spectrum is concentrated in fewer peaks, and the superimposed spectra reveal a clear redistribution of the spectral content.

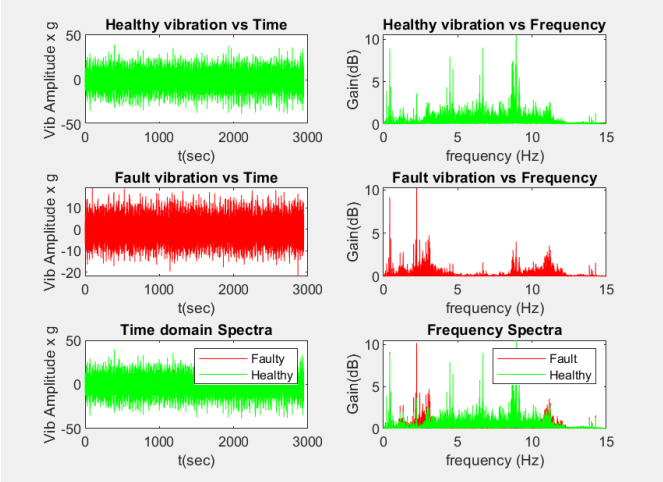


Fig. 4. Automatically generated plots for the gear fault case: healthy (green) and faulty (red) vibration amplitude versus time (left column) and the corresponding frequency spectra (right column), with the superimposed comparisons in the bottom row.

The extracted features are reported in the information box shown in Fig. 5 and summarized in Table III. The faulty signal has an RMS of 3.554 compared with 5.957 for the healthy signal, giving R = 0.597. The dominant peaks occur at an order of 0.2×RPM. The number of significant peaks decreases from seven in the healthy spectrum to four in the faulty spectrum; two peak frequencies match, but none match in serial position.

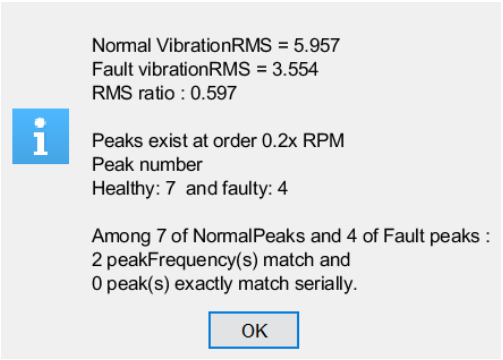


Fig. 5. Feature report generated by the software for the gear fault case.

TABLE III
EXTRACTED FEATURES FOR THE GEAR FAULT CASE

Feature	Healthy	Faulty
RMS	5.957	3.554
RMS ratio, R	—	0.597
Peak order, M	—	0.2×RPM
Number of peaks, N	7	4
Matching peak frequencies, C_f	—	2
Exact serial matches, C_e	—	0

Evaluating the rules in Table II, the bearing prerequisite fails because $R < 1$ and $M < 2$, and the mechanical fault condition fails because R lies outside the range 0.98–1.1. The gear fault condition, $M < 0.7$ and $R < 0.7$, is satisfied, which is consistent with the gear fault characteristics in Table I (order ≤ 0.7 and lower relative RMS). Accordingly, the software automatically declared a gear fault, as shown in Fig. 6, without requiring any manual interpretation of the plots.

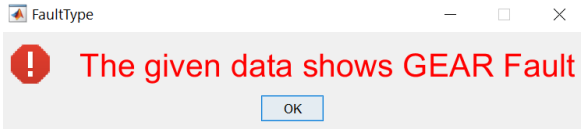


Fig. 6. Automatic fault declaration produced by the software for the gear fault case.

VI. DISCUSSION

A. Significance

Vibration processing in the time and frequency domains, power spectral density estimation, and related feature extraction techniques have been widely used by experts; however, the identification of the fault type has conventionally remained a manual task. The principal novelty of the proposed method lies in closing this loop: after completing the vibration analysis, the computer itself declares the type of fault. This relieves maintenance personnel of manual spectral interpretation and represents a step toward automated prediction systems for machine health. Beyond laboratory and industrial use, the approach, once refined with further experimental statistics, can support the simultaneous monitoring of large numbers of machines, locate faults in locations that are inaccessible to humans, and provide early warning of massive machine failures, thereby saving both cost and lives.

B. Limitations and Future Work

The rule base was derived for a specific type of machine operating at a fixed RPM and a particular loading condition, and its thresholds are therefore not directly transferable to other machines or operating points. To overcome this limitation, data from multi-axial accelerometers should be processed and combined, which would also enable the discrimination of unbalance from horizontal and vertical misalignment. Furthermore, the FFT assumes stationarity and does not provide time localization of spectral events; these drawbacks can be addressed by employing the short-time Fourier transform (STFT) or, where that is insufficient, the wavelet transform (WT).

VII. CONCLUSION

This paper presented an automated fault detection method for rotating machines based on the comparison of accelerometer vibration signals from a healthy and a test machine. By combining time-domain RMS analysis, FFT-based spectral analysis, peak order calculation, spectral peak counting, and peak frequency and phase matching with an empirically derived rule base, the method automatically declares inner race, outer race, and ball bearing faults, mechanical faults, and gear faults. Its MATLAB implementation requires minimal user input and reports both the diagnosis and the supporting evidence in graphical and textual form. A representative case study demonstrated the automatic identification of a gear fault. Future work will extend the method to multi-axial measurements and time-frequency representations to improve its generality across machines and operating conditions.

ACKNOWLEDGMENT

This work was carried out as a project of the course MATH 4421: Random Signal Processing at the Islamic University of Technology (IUT).

REFERENCES

[1] NASA Technical Reports Server, Document no. 19970034695. [Online]. Available: https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/19970034695.pdf

[2] *IEEE Recommended Practice for the Design of Reliable Industrial and Commercial Power Systems (IEEE Gold Book)*, IEEE Std 493-1997, Appendix H, 1997.

[3] P. F. Albrecht, J. C. Appiarius, and R. M. McCoy, "Assessment of the reliability of motors in utility applications—Updated," *IEEE Trans. Energy Convers.*, vol. EC-1, no. 1, pp. 39–46, 1986.

APPENDIX: MATLAB SOURCE CODE

The complete MATLAB implementation of the proposed method is listed below.

A. Main Diagnostic Script (main_fault_detector.m)

```
% Automated Fault Detection for Rotating Machinery via Vibration Signal Analysis
% Author: Mohammad Mahmudul Hasan et al.
% Course: Random Signal Processing (MATH 4421), Department of EEE, IUT

clear;
close all;
clc;

%% 1. Interactive Data Import via Graphical Dialog
disp('Select the .mat file containing Baseline (Healthy) Machine Vibration Data...');
pause(1);
uiopen('load');
varsHealthy = who;
disp('Workspace variables loaded:');
disp(varsHealthy);
rawSignalHealthy = input('Enter the variable name for Healthy Vibration Data: ');

disp('Select the .mat file containing Test (Potentially Faulty) Machine Vibration Data...');
pause(1);
uiopen('load');
varsFaulty = who;
disp('Workspace variables loaded:');
disp(varsFaulty);
rawSignalTest = input('Enter the variable name for Test Vibration Data: ');

%% 2. Temporal Alignment and Sampling Configuration
isDurationKnown = input('Is the signal acquisition duration known? (Y/N): ', 's');

if strcmpi(isDurationKnown, 'Y')
    durationSec = input('Enter signal acquisition duration (seconds): ');
    Fs = input('Enter sampling frequency Fs (Hz): ');
    totalSamples = round(Fs * durationSec);
    timeAxisHealthy = (0:totalSamples-1) / Fs;
    timeAxisFaulty  = (0:totalSamples-1) / Fs;
    sigHealthy = rawSignalHealthy(1:totalSamples);
    sigFaulty  = rawSignalTest(1:totalSamples);
    analysisLen = totalSamples;
else
    lenTest    = length(rawSignalTest);
    lenHealthy = length(rawSignalHealthy);
    analysisLen = min(lenTest, lenHealthy);
    Fs = input('Enter sampling frequency Fs (Hz): ');
    durationSec = analysisLen / Fs;

    timeAxisHealthy = (0:analysisLen-1) / Fs;
    timeAxisFaulty  = (0:analysisLen-1) / Fs;
    sigHealthy = rawSignalHealthy(1:analysisLen);
    sigFaulty  = rawSignalTest(1:analysisLen);
end

%% 3. Multi-Domain Signal Analysis and Feature Extraction
[orderRatioM, freqAxisHealthy, specHealthy, specFaulty, ...
 valPeaksHealthy, valPeaksFaulty, peakFreqsHealthy, peakFreqsFaulty, ...
 numPeaksHealthy, numPeaksFaulty] = ...
    detectVibrationFeatures(sigHealthy, sigFaulty, analysisLen, Fs, timeAxisHealthy, timeAxisFaulty);

%% 4. Statistical Energy and Root-Mean-Square (RMS) Metrics
rmsHealthy = rms(sigHealthy);
rmsFaulty  = rms(sigFaulty);
rmsRatio   = rmsFaulty / rmsHealthy;

isEnergyElevated = (rmsFaulty > rmsHealthy);

%% 5. Spectral Peak Frequency and Order Coherence Matching
countFreqMatches  = 0;  % Number of coincident peak frequencies (within 1 Hz)
countSerialMatches = 0; % Number of identically ranked coincident peak frequencies
```

```
for i = 1:numPeaksHealthy
    for j = 1:numPeaksFaulty
        if round(peakFreqsHealthy(1, i)) == round(peakFreqsFaulty(1, j))
            countFreqMatches = countFreqMatches + 1;
            if i == j
                countSerialMatches = countSerialMatches + 1;
            end
        end
    end
end

%% 6. Graphical Summary of Diagnostic Parameters
summaryReport = sprintf([ ...
    '\\bfDiagnostic Feature Summary:\\rm\\n' ...
    'Baseline (Healthy) Vibration RMS: %.3f g\\n' ...
    'Test Vibration RMS: %.3f g\\n' ...
    'RMS Severity Ratio (R): %.3f\\n\\n' ...
    'Dominant Fault Order: %.2fx RPM\\n' ...
    'Detected Significant Peaks -> Healthy: %d | Test: %d\\n\\n' ...
    'Spectral Matching Results:\\n' ...
    ' - Coincident Frequencies: %d\\n' ...
    ' - Exact Serial/Phase Matches: %d'], ...
    rmsHealthy, rmsFaulty, rmsRatio, orderRatioM, ...
    numPeaksHealthy, numPeaksFaulty, countFreqMatches, countSerialMatches);

helpdlg(summaryReport, 'Vibration Feature Extraction Summary');

%% 7. Rule-Based Inference Engine for Fault Classification
isBearingFault    = false;
isInnerRaceFault  = false;
isOuterRaceFault  = false;
isBallFault       = false;
isGearFault       = false;
isMechanicalFault = false;

faultCount = 0;

% --- A. Bearing Fault Evaluation ---
% Prerequisite: elevated RMS energy, complete loss of baseline spectral peak coherence,
% high-order excitation (>2x RPM), and minimum 2x RMS amplification.
if (isEnergyElevated && countFreqMatches == 0 && countSerialMatches == 0 && ...
    orderRatioM > 2.0 && round(rmsRatio) >= 2)
    isBearingFault = true;
end

if isBearingFault
    % 1. Ball Bearing Defect
    if round(rmsRatio) == 2
        isBallFault = true;
        faultCount = faultCount + 1;
    end

    % 2. Inner Race Defect
    if (((orderRatioM >= 2.0 && orderRatioM <= 3.0) && round(rmsRatio) >= 4) || ...
        ((orderRatioM >= 10.0 && orderRatioM <= 18.0) && round(rmsRatio) >= 4))
        isInnerRaceFault = true;
        faultCount = faultCount + 1;
    end

    % 3. Outer Race Defect
    if (((orderRatioM >= 3.0 && orderRatioM <= 4.0) && (rmsRatio >= 3.0 && rmsRatio <= 5.5)) || ...
        ((orderRatioM >= 18.8) && (rmsRatio >= 3.0 && rmsRatio <= 5.5)))
        isOuterRaceFault = true;
        faultCount = faultCount + 1;
    end
end

% --- B. Mechanical Faults (Unbalance / Misalignment) ---
if (orderRatioM >= 0.1 && orderRatioM <= 2.5) && (rmsRatio >= 0.98 && rmsRatio <= 1.1)
    isMechanicalFault = true;
    faultCount = faultCount + 1;
end

% --- C. Gear Fault Evaluation ---
```

```
if (orderRatioM < 0.7 && rmsRatio < 0.7) || (numPeaksHealthy == 0 && numPeaksFaulty == 0)
    isGearFault = true;
    faultCount = faultCount + 1;
end

%% 8. Automated Classification Declaration Modal
dialogStyle.Interpreter = 'tex';
dialogStyle.WindowStyle = 'modal';

diagnosticMessages = cell(max(faultCount, 1), 1);
msgIdx = 0;

if isInnerRaceFault
    msgIdx = msgIdx + 1;
    diagnosticMessages{msgIdx} = '\fontsize{14} \color{red} \bfFAULT DETECTED: Inner Race Bearing Defect\rm';
end

if isOuterRaceFault
    msgIdx = msgIdx + 1;
    diagnosticMessages{msgIdx} = '\fontsize{14} \color{red} \bfFAULT DETECTED: Outer Race Bearing Defect\rm';
end

if isBallFault
    msgIdx = msgIdx + 1;
    diagnosticMessages{msgIdx} = '\fontsize{14} \color{red} \bfFAULT DETECTED: Ball Bearing Element Defect\rm';
end

if isGearFault
    msgIdx = msgIdx + 1;
    diagnosticMessages{msgIdx} = '\fontsize{14} \color{red} \bfFAULT DETECTED: Gear Mesh / Tooth Defect\rm';
end

if isMechanicalFault
    msgIdx = msgIdx + 1;
    diagnosticMessages{msgIdx} = ['\fontsize{14} \color{red} \bfFAULT DETECTED: Mechanical Fault (Unbalance /
Misalignment)\rm\n' ...
                                  '\fontsize{10} \color{black} (Examine secondary multi-axial sensor channels to
isolate orientation)'];
end

if faultCount == 0
    diagnosticMessages{1} = '\fontsize{14} \color{green} \bfSTATUS NORMAL: No Known Fault Pattern Identified\rm';
end

msgbox(diagnosticMessages, 'Automated Fault Diagnosis Result', 'warn', dialogStyle);
```

B. Function detectVibrationFeatures.m

```
function [orderRatioM, freqAxisHealthy, specHealthy, specFaulty, ...
          valPeaksHealthy, valPeaksFaulty, peakFreqsHealthy, peakFreqsFaulty, ...
          numPeaksHealthy, numPeaksFaulty] = ...
          detectVibrationFeatures(sigHealthy, sigFaulty, analysisLen, Fs, timeAxisHealthy, timeAxisFaulty)
% Analyzes time/frequency spectra, extracts orders, and detects significant peaks.

figure('Name', 'Vibration Analysis: Time and Frequency Domain Profiles', 'NumberTitle', 'off');

% 1. Time-Domain Waveforms
subplot(3, 2, 1);
plot(timeAxisHealthy, sigHealthy, 'g');
title('Healthy Baseline Vibration vs Time');
xlabel('Time (s)'); ylabel('Acceleration (g)'); grid on;

subplot(3, 2, 3);
plot(timeAxisFaulty, sigFaulty, 'r');
title('Test Signal Vibration vs Time');
xlabel('Time (s)'); ylabel('Acceleration (g)'); grid on;

subplot(3, 2, 5);
plot(timeAxisFaulty, sigFaulty, 'r', timeAxisHealthy, sigHealthy, 'g');
legend('Test (Faulty)', 'Baseline (Healthy)', 'Location', 'northeast');
title('Superimposed Time-Domain Signatures');
xlabel('Time (s)'); ylabel('Acceleration (g)'); grid on;

%% 2. Single-Sided Amplitude Spectrum via FFT
```

```
[freqAxisHealthy, specHealthy] = computeSingleSidedFFT(sigHealthy, analysisLen, Fs);
[~,               specFaulty]  = computeSingleSidedFFT(sigFaulty,  analysisLen, Fs);

subplot(3, 2, 2);
plot(freqAxisHealthy, specHealthy, 'g');
title('Healthy Baseline Spectrum');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); grid on;

subplot(3, 2, 4);
plot(freqAxisHealthy, specFaulty, 'r');
title('Test Signal Spectrum');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); grid on;

subplot(3, 2, 6);
plot(freqAxisHealthy, specFaulty, 'r', freqAxisHealthy, specHealthy, 'g');
legend('Test (Faulty)', 'Baseline (Healthy)', 'Location', 'northeast');
title('Superimposed Frequency Spectra');
xlabel('Frequency (Hz)'); ylabel('Gain (dB)'); grid on;

%% 3. Rotational Order Analysis
[peakOrdersHealthy] = extractDominantOrders(specHealthy, freqAxisHealthy, 2);
[peakOrdersFaulty]  = extractDominantOrders(specFaulty,  freqAxisHealthy, 2);

orderRatioVector = peakOrdersFaulty ./ peakOrdersHealthy;
orderRatioM      = mean(orderRatioVector);

%% 4. Peak Detection and Salience Thresholding
thresholdAdaptive = max(specHealthy) / pi;

peakFinder = dsp.PeakFinder('PeakType', 'Maxima', ...
                            'PeakIndicesOutputPort', true, ...
                            'PeakValuesOutputPort', true, ...
                            'IgnoreSmallPeaks', true, ...
                            'PeakThreshold', thresholdAdaptive);

[numPeaksHealthy, idxHealthy, ~] = peakFinder(specHealthy);
[valPeaksHealthy, ~, peakFreqsHealthy] = extractRankedMaxima(specHealthy, numPeaksHealthy, freqAxisHealthy);

[numPeaksFaulty, idxFaulty, ~]   = peakFinder(specFaulty);
[valPeaksFaulty, ~, peakFreqsFaulty]   = extractRankedMaxima(specFaulty, numPeaksFaulty, freqAxisHealthy);

end
```

C. Function computeSingleSidedFFT.m

```
function [freqAxis, singleSidedSpectrum] = computeSingleSidedFFT(signalInput, analysisLen, Fs)
% Computes the scaled single-sided discrete Fourier amplitude spectrum.

dftCoeffs = fft(signalInput);
twoSidedSpectrum = (abs(dftCoeffs / analysisLen)) * 10;

singleSidedSpectrum = twoSidedSpectrum(1 : floor(analysisLen/2) + 1);
singleSidedSpectrum(2 : end-1) = 2 * singleSidedSpectrum(2 : end-1);

freqAxis = Fs * (0 : floor(analysisLen/2)) / analysisLen;
end
```

D. Function extractDominantOrders.m

```
function [peakOrders] = extractDominantOrders(spectrumInput, freqAxis, numOrders)
% Extracts dominant spectral orders relative to fundamental shaft speed.

[orderSpec, orderGrid] = orderspectrum(spectrumInput, freqAxis);
[~, peakOrders] = findpeaks(orderSpec, orderGrid, 'SortStr', 'descend', 'NPeak', numOrders);
peakOrders = round(peakOrders, 3);
end
```

E. Function extractRankedMaxima.m

```
function [sortedAmps, sortedIndices, sortedFreqs] = extractRankedMaxima(spectrumInput, numPeaks, freqAxis)
% Iteratively extracts and ranks the top N spectral peaks by amplitude.

sortedAmps    = zeros(1, numPeaks);
sortedIndices = zeros(1, numPeaks);
```

```
sortedFreqs   = zeros(1, numPeaks);

tempSpectrum = spectrumInput;

for i = 1:numPeaks
    [sortedAmps(i), sortedIndices(i)] = max(tempSpectrum);
    tempSpectrum(sortedIndices(i)) = -Inf; % Suppress current peak to find next
end

for j = 1:numPeaks
    sortedFreqs(j) = freqAxis(sortedIndices(j));
end

end
```